

SQL Interview Notes

Authored by : Arpit Raj , Lnmiit jaipur

1. Employees Earning More Than Their Managers

Table: Employee

Column Name	Type
id	int
name	varchar
salary	int
managerId	int

id is the primary key (column with unique values) for this table.

Each row of this table indicates the ID of an employee, their name, salary, and the ID of their manager.

Write a solution to find the employees who earn more than their managers.

Return the result table in any order.

Example 1:

Input:

Employee table:

id	name	salary	managerId
1	Joe	70000	3
2	Henry	80000	4
3	Sam	60000	Null
4	Max	90000	Null

Output:

Employee
Joe

Explanation: Joe is the only employee who earns more than his manager.

```
# Write your MySQL query statement below
SELECT
    e.name AS Employee
FROM Employee e
JOIN Employee m
ON e.managerId = m.id
WHERE e.salary > m.salary;
```

2. Duplicate Emails

Table: Person

Column Name	Type
id	int
email	varchar

id is the primary key (column with unique values) for this table.

Each row of this table contains an email. The emails will not contain uppercase letters.

Write a solution to report all the duplicate emails. Note that it's guaranteed that the email field is not NULL.

Return the result table in any order.

```
# Write your MySQL query statement below
SELECT
    email AS Email
FROM Person
GROUP BY email
HAVING COUNT(*) > 1;
```

3. Department Highest Salary

Table: Employee

Column Name	Type
id	int
name	varchar
salary	int
departmentId	int

id is the primary key (column with unique values) for this table.

departmentId is a foreign key (reference columns) of the ID from the Department table.

Each row of this table indicates the ID, name, and salary of an employee. It also contains the ID of their department.

Table: Department

Column Name	Type
id	int
name	varchar

id is the primary key (column with unique values) for this table. It is guaranteed that department name is not NULL.

Each row of this table indicates the ID of a department and its name.

Write a solution to find employees who have the highest salary in each of the departments.

Return the result table in any order.

Example 1:

Input:

Employee table:

id	name	salary	departmentId
1	Joe	70000	1
2	Jim	90000	1
3	Henry	80000	2
4	Sam	60000	2
5	Max	90000	1

Department table:

id	name
1	IT
2	Sales

Output:

Department	Employee	Salary
IT	Jim	90000
Sales	Henry	80000
IT	Max	90000

Explanation: Max and Jim both have the highest salary in the IT department and Henry has the highest salary in the Sales department.

```
SELECT
    d.name AS Department,
    e.name AS Employee,
    e.salary AS Salary
FROM Employee e
JOIN Department d
ON e.departmentId = d.id
JOIN
(
    SELECT
        departmentId,
        MAX(salary) AS max_salary
    FROM Employee
    GROUP BY departmentId
) t
ON e.departmentId = t.departmentId
AND e.salary = t.max_salary;
```

4. Trips and Users

Table: Trips

Column Name	Type
id	int
client_id	int
driver_id	int
city_id	int
status	enum
request_at	varchar

id is the primary key (column with unique values) for this table.

The table holds all taxi trips. Each trip has a unique id, while client_id and driver_id are foreign keys to the users_id at the Users table.

Status is an ENUM (category) type of ('completed', 'cancelled_by_driver', 'cancelled_by_client').

Table: Users

Column Name	Type
users_id	int
banned	enum
role	enum

users_id is the primary key (column with unique values) for this table.

The table holds all users. Each user has a unique users_id, and role is an ENUM type of ('client', 'driver', 'partner').

banned is an ENUM (category) type of ('Yes', 'No').

The cancellation rate is computed by dividing the number of canceled (by client or driver) requests with unbanned users by the total number of requests with unbanned users on that day.

Write a solution to find the cancellation rate of requests with unbanned users (both client and driver must not be banned) each day between "2013-10-01" and "2013-10-03" with at least one trip. Round Cancellation Rate to two decimal points.

Let's break it down.

Task: For every day between 2013-10-01 and 2013-10-03 find Cancellation Rate where:
Cancellation Rate = Cancelled Trips / Total Valid Trips

Only count trips where BOTH:

- client is not banned
- driver is not banned

Example: Suppose

Trip	Status	Client	Driver
1	completed	Not banned	Not banned
2	cancelled_by_driver	Not banned	Not banned
3	completed	Banned	Not banned

Trip 3 is ignored completely.

Remaining:

Total valid trips: 2

Cancelled trips: 1

Cancellation Rate: $1 / 2 = 0.50$

```
# Write your MySQL query statement below
SELECT
    t.request_at AS Day,
    ROUND(
        AVG(
            CASE
                WHEN t.status = 'completed' THEN 0
                ELSE 1
            END
        ),
        2
    ) AS 'Cancellation Rate'
FROM Trips t
JOIN Users c
ON t.client_id = c.users_id
JOIN Users d
ON t.driver_id = d.users_id
WHERE c.banned = 'No'
AND d.banned = 'No'
AND t.request_at BETWEEN '2013-10-01' AND '2013-10-03'
GROUP BY t.request_at;
```

5. Investments in 2016

Table: Insurance

Column Name	Type
pid	int
tiv_2015	float
tiv_2016	float
lat	float
lon	float

pid is the primary key (column with unique values) for this table.

Each row of this table contains information about one policy where:

pid is the policyholder's policy ID.

tiv_2015 is the total investment value in 2015 and tiv_2016 is the total investment value in 2016.

lat is the latitude of the policy holder's city. It's guaranteed that lat is not NULL.

lon is the longitude of the policy holder's city. It's guaranteed that lon is not NULL.

Write a solution to report the sum of all total investment values in 2016 tiv_2016, for all policyholders who:

- have the same tiv_2015 value as one or more other policyholders, and
- are not located in the same city as any other policyholder (i.e., the (lat, lon) attribute pairs must be unique).

Round tiv_2016 to two decimal places.

Example 1:

Input:

Insurance table:

pid	tiv_2015	tiv_2016	lat	lon
1	10	5	10	10
2	20	20	20	20
3	10	30	20	20
4	10	40	40	40

Output:

tiv_2016
45.00

Explanation:

The first record in the table, like the last record, meets both of the two criteria.

The tiv_2015 value 10 is the same as the third and fourth records, and its location is unique.

The second record does not meet any of the two criteria. Its tiv_2015 is not like any other policyholders and its location is the same as the third record, which makes the third record fail, too.

So, the result is the sum of tiv_2016 of the first and last record, which is 45.

```
SELECT
    ROUND(SUM(i.tiv_2016),2) AS tiv_2016
FROM Insurance i
JOIN
(
    SELECT tiv_2015
    FROM Insurance
    GROUP BY tiv_2015
    HAVING COUNT(*) > 1
) t1
ON i.tiv_2015 = t1.tiv_2015
JOIN
(
    SELECT lat, lon
    FROM Insurance
    GROUP BY lat, lon
    HAVING COUNT(*) = 1
) t2
ON i.lat = t2.lat
AND i.lon = t2.lon;
```

6. Classes More Than 5 Students

Table: Courses

Column Name	Type
student	varchar
class	varchar

(student, class) is the primary key (combination of columns with unique values) for this table.
Each row of this table indicates the name of a student and the class in which they are enrolled.

Write a solution to find all the classes that have at least five students.
Return the result table in any order.

Example 1:

Input:

Courses table:

student	class
A	Math
B	English
C	Math
D	Biology
E	Math
F	Computer
G	Math
H	Math
I	Math

Output:

class
Math

```
SELECT
    class
FROM Courses
GROUP BY class
HAVING COUNT(student) >= 5;
```

7. Friend Requests II: Who Has the Most Friends

Table: RequestAccepted

Column Name	Type
requester_id	int
accepter_id	int
accept_date	date

(requester_id, accepter_id) is the primary key (combination of columns with unique values) for this table.

This table contains the ID of the user who sent the request, the ID of the user who received the request, and the date when the request was accepted.

Write a solution to find the people who have the most friends and the most friends number.

The test cases are generated so that only one person has the most friends.

Example 1:

Input:

RequestAccepted table:

requester_id	accepter_id	accept_date
1	2	2016/06/03
1	3	2016/06/08
2	3	2016/06/08
3	4	2016/06/09

Output:

id	num
3	3

Explanation:

The person with id 3 is a friend of people 1, 2, and 4, so he has three friends in total, which is the most number than any others.

Key Observation: Every friendship appears once in the table.

Example: |1|2| means 1 is friend of 2 AND 2 is friend of 1. So we must count both people.

```
SELECT
    id,
    COUNT(*) AS num
FROM
    (
        SELECT requester_id AS id
        FROM RequestAccepted
        UNION ALL
        SELECT acceptor_id
        FROM RequestAccepted
    ) t
GROUP BY id
ORDER BY num DESC
LIMIT 1;
```

8. Tree Node

Table: Tree

Column Name	Type
id	int
p_id	int

id is the column with unique values for this table.

Each row of this table contains information about the id of a node and the id of its parent node in a tree. The given structure is always a valid tree.

Each node in the tree can be one of three types:

"Leaf": if the node is a leaf node.

"Root": if the node is the root of the tree.

"Inner": If the node is neither a leaf node nor a root node.

Write a solution to report the type of each node in the tree.

Return the result table in any order.

Example 1:

Input:

Tree table:

id	p_id
1	null
2	1
3	1
4	2
5	2

Output:

id	type
1	Root
2	Inner
3	Leaf
4	Leaf
5	Leaf

Key Observation:

To know whether a node is a leaf, ask: Does anyone have me as their parent?

In SQL: `id IN (SELECT p_id FROM Tree)`

If yes → It has children (Inner).

```
SELECT
  id,
  CASE
    WHEN p_id IS NULL THEN 'Root'
    WHEN id IN (SELECT p_id FROM Tree) THEN 'Inner'
    ELSE 'Leaf'
  END AS type
FROM Tree;
```

9. Product Sales Analysis

Table: Sales

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

(sale_id, year) is the primary key (combination of columns with unique values) of this table.

Each row records a sale of a product in a given year.

A product may have multiple sales entries in the same year.

Note that the per-unit price.

Write a solution to find all sales that occurred in the first year each product was sold.

For each product_id, identify the earliest year it appears in the Sales table.

Return all sales entries for that product in that year.

```
# Write your MySQL query statement below
SELECT
    s.product_id,
    s.year AS first_year,
    s.quantity,
    s.price
FROM Sales s
JOIN
(
    SELECT
        product_id,
        MIN(year) AS first_year
    FROM Sales
    GROUP BY product_id
) t
ON s.product_id = t.product_id
AND s.year = t.first_year;
```

10. Average Selling Price

Table: Prices

Column Name	Type
product_id	int
start_date	date
end_date	date
price	int

(product_id, start_date, end_date) is the primary key (combination of columns with unique values) for this table.

Each row of this table indicates the price of the product_id in the period from start_date to end_date. For each product_id there will be no two overlapping periods.

Table: UnitsSold

Column Name	Type
product_id	int
purchase_date	date
units	int

This table may contain duplicate rows.

Each row of this table indicates the date, units, and product_id of each product sold.

Write a solution to find the average selling price for each product. average_price should be rounded to 2 decimal places. If a product does not have any sold units, its average selling price is assumed to be 0.

Return the result table in any order.

Example 1:

Input:

Prices table:

product_id	start_date	end_date	price
1	2019-02-17	2019-02-28	5
1	2019-03-01	2019-03-22	20
2	2019-02-01	2019-02-20	15
2	2019-02-21	2019-03-31	30

UnitsSold table:

product_id	purchase_date	units
1	2019-02-25	100
1	2019-03-01	15
2	2019-02-10	200
2	2019-03-22	30

Output:

product_id	average_price
1	6.96
2	16.96

Formula: *Average Price* = $\Sigma(\text{price} \times \text{units}) / \Sigma(\text{units})$

Example for 100 units at price 5 & 50 units at price 20:

cost = $100 \times 5 = 500$

cost = $50 \times 20 = 1000$

Total money = 1500, Total units = 150

Average = $1500 / 150 = 10$

```

SELECT
    p.product_id,
    ROUND(
        CASE
            WHEN SUM(u.units) IS NULL THEN 0
            ELSE SUM(p.price * u.units) / SUM(u.units)
        END,
        2
    ) AS average_price
FROM Prices p
LEFT JOIN UnitsSold u
ON p.product_id = u.product_id
AND u.purchase_date BETWEEN p.start_date AND p.end_date
GROUP BY p.product_id;

```

11. Movie Rating

Table: Movies

Column Name	Type
movie_id	int
title	varchar

movie_id is the primary key (column with unique values) for this table.

title is the name of the movie. Each movie has a unique title.

Table: Users

Column Name	Type
user_id	int
name	varchar

Table: MovieRating

Column Name	Type
movie_id	int
user_id	int
rating	int
created_at	date

(movie_id, user_id) is the primary key (column with unique values) for this table.

This table contains the rating of a movie by a user in their review.

created_at is the user's review date.

Write a solution to:

Find the name of the user who has rated the greatest number of movies. In case of a tie, return the lexicographically smaller user name.

Find the movie name with the highest average rating in February 2020. In case of a tie, return the lexicographically smaller movie name.

Example 1:

Input:

Output:

results
Daniel
Frozen 2

Part 1 — User with most reviews: Daniel has 2 reviews (user 1).

Part 2 — Movie with highest average rating in Feb 2020: Movie 1 (Frozen 2) has highest average rating.

```

SELECT results
FROM
(
    SELECT
        u.name AS results
    FROM Users u
    JOIN MovieRating mr
    ON u.user_id = mr.user_id
    GROUP BY u.user_id, u.name
    ORDER BY COUNT(*) DESC, u.name
    LIMIT 1
) a

UNION ALL

SELECT results
FROM
(
    SELECT
        m.title AS results
    FROM Movies m
    JOIN MovieRating mr
    ON m.movie_id = mr.movie_id
    WHERE mr.created_at >= '2020-02-01'
    AND mr.created_at < '2020-03-01'
    GROUP BY m.movie_id, m.title
    ORDER BY AVG(mr.rating) DESC, m.title
    LIMIT 1
) b;

```

12. Register Percentage

Table: Users

Column Name	Type
user_id	int
user_name	varchar

Table: Register

Column Name	Type
contest_id	int
user_id	int

(contest_id, user_id) is the primary key for this table.

Each row of this table contains the id of a user and the contest they registered into.

Write a solution to find the percentage of the users registered in each contest rounded to two decimals. Return the result table ordered by percentage in descending order. In case of a tie, order it by contest_id in ascending order.

Example 1 Output:

contest_id	percentage
208	100.0
209	100.0
210	100.0
215	66.67
207	33.33

Formula:

Percentage = (Number of registered users / Total users) × 100

```
SELECT
    r.contest_id,
    ROUND(COUNT(r.user_id) * 100 /
          (SELECT COUNT(*) FROM Users), 2) AS percentage
FROM Register r
GROUP BY r.contest_id
ORDER BY percentage DESC, r.contest_id;
```

13. Salary Categories

Table: Accounts

Column Name	Type
account_id	int
income	int

account_id is the primary key for this table.

Each row contains information about the monthly income for one bank account.

Write a solution to calculate the number of bank accounts for each salary category. The salary categories are:

"Low Salary": All the salaries strictly less than \$20000.

"Average Salary": All the salaries in the inclusive range [\$20000, \$50000].

"High Salary": All the salaries strictly greater than \$50000.

The result table must contain all three categories. If there are no accounts in a category, return 0.

Example 1 Output:

category	accounts_count
Low Salary	1
Average Salary	0
High Salary	3

Explanation:

Low Salary (income < 20000): Account 2.

Average Salary (20000 <= income <= 50000): No accounts.

High Salary (income > 50000): Accounts 3, 6, and 8.